

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_3512746c-54e\agent-a66a6864591e72ef8.jsonl`
- SHA-256: `20bcc50c7914ee31da85bc55f9bbcb094d059ff6b061665f45408d523b65d51`
- Source modified: `2026-07-06T09:13:45+00:00`
- Imported at: `2026-07-08T16:00:05+00:00`
- project: `wf_3512746c-54e`
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

Transcript

1. user (2026-07-06T09:07:16.618Z)

Reviewing GitHub PR #39 of the repo at C:/Users/avidu/Projects/Telegram (branch feat/track-e-selective-scan -> main, one commit db87cc0).

It implements ADR 0009 Track E: selective source scanning ? /watch, /unwatch, /channels, /watchlist commands +

an ingestion gate in telethon_client.py that skips unwatched sources. Changed files ONLY: command_service.py (new /watch etc + _resolve_source), commands.py (re-export patterns), db.py (set_source_watched, list_account_sources), telethon_client.py (the gate), tests/test_command_service.py (10 tests).

Diff +254/-3. READ docs/adr/0009-selective-channel-scanning.md ? it is the spec this must conform to.

KEY ADR 0009 REQUIREMENTS (quote/verify against the code):

- Line 37-38: "The default is locked as opt-in: DEFAULT 0, so a freshly-onboarded user scans nothing until they opt in."
- Line 44: "...return early if watched is false, before any dedup or filter work."
- Line 130-131: "New channels discovered after the migration default to watched = 0 and must be explicitly watched."
- Line 48-54 (discoverability): lazy-populated rows should be created watched=0 so they appear in /channels for opt-in;
/channels should also upsert candidates from client.get_dialogs().
- Line 64: "/watch <#name|id>" ? resolve by channel name/username OR id.

Ground rules:

- READ actual code (Read/Grep), cite file:line. You MAY run repros with C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe (build schema via db._apply_schema on aiosqlite ":memory:"); seed via db.upsert_account + db.upsert_source; drive command_service.CommandService().handle()).
- NOTE: real telegram sources have a NUMERIC provider_source_id (str(channel_id) e.g. '-1001234567890'); upsert_channel creates them with watched=True. The on_new_message gate is inside a '# pragma: no cover' function.
- Authoritative gate ALREADY RUN (don't re-run full suite): ruff check + format clean; pytest 492 passed @ 97.10% (floor 80%).
- Report ONLY defects in THIS PR's diff. Deviation from the ACCEPTED ADR 0009 is a high-value finding. No praise.
Concrete failure scenario per finding, ranked most-severe first, honest severity.
- Severity: blocker (feature broken / data loss on the live path), high (real bug or ADR-contract violation users hit),
medium (edge/latent/half-broken feature), low (hygiene/UX/discoverability).

DIMENSION: ADR 0009 conformance of the ingestion gate + new-source default (telethon_client.py:184-191 + db.upsert_channel).

SEED to verify/refute (I believe CONFIRMED, high): the gate only skips when a source row EXISTS

and watched=False
(if src is None it falls through). A brand-NEW channel (no source row) is therefore processed (scanned/dedup/deliver),
and later upsert_channel (db.py) creates it with watched=TRUE. So new channels are auto-watched (OPT-OUT), directly
violating ADR 0009 which locks opt-in DEFAULT 0 and says "new channels ... default to watched = 0 and must be explicitly watched" and "return early ... before any dedup or filter work". Trace the exact on_new_message ordering (gate vs upsert_channel), confirm upsert_channel's watched value, and assess: does a freshly-onboarded user actually "scan nothing until they opt in" (ADR) or does every new channel get auto-scanned+auto-watched? Also: is the first message from a new channel delivered (should it be skipped)? Report the ADR deviation and its severity.

2. assistant (2026-07-06T09:07:19.275Z)

I'll review this PR against ADR 0009. Let me start by reading the ADR spec and the changed files.

3. user (2026-07-06T09:07:19.835Z)

```
1      # ADR 0009: Selective channel scanning and multiplexed routing
2
3      - **Status**: Accepted
4      - **Date**: 2026-07-04
5      - **Decision owner**: project owner
6
7      ## Context
8
9      Today the userbot scans every channel/group the account is a member of.
10     `manager.py` registers `events.NewMessage(incoming=True)` with no `chats=`
11     filter, and `on_new_message` processes every video regardless of source. The
12     `channels` table exists but is populated lazily as a record of seen channels;
13     it is not used as a filter.
14
15     The product vision is a multiplexing video re-distribution system:
16
17     - A small set of high-quality source channels (e.g. spiritual, classical
18       music) are scanned for incoming videos.
19     - Matches are routed to themed destination broadcast channels (e.g. a
20       Reggae channel, a Rap channel) that subscribers opt into.
21     - A user therefore subscribes to outputs by topic, not to inputs by source.
22
23     This requires two capabilities the current architecture does not have:
24
25     1. Selective scanning ? choose which source channels are watched, rather
26       than ingesting all of them.
27     2. Per-source ? per-destination routing ? a match from source channel X is
28       delivered to a specific destination channel Y, instead of the single global
29       `delivery_target` (ADR 0005).
30
31     ## Decision
32
33     Add three additive, independently-shippable capabilities:
34
35     ### 1. Selective scanning (watched-flag)
36
37     Add a `watched` flag to the `channels` table. The default is locked as
38     opt-in: `DEFAULT 0`, so a freshly-onboarded user scans nothing until they
39     explicitly `/watch` a channel. This matches the multiplexing product shape
40     (sources are a curated set) and avoids blindly re-ingesting every chat the
41     account is a member of.
42
```

```

43 Gate `on_new_message` at the top: after computing `channel_id`, look up the
44 row and `return` early if `watched` is false, before any dedup or filter work.
45
46 This is a small migration plus ~5 lines in `telethon_client.py`.
47
48 **Discoverability note.** With `DEFAULT 0` plus the current lazy population
49 (a `channels` row is only inserted when a message is first seen), opt-in is
50 necessary but not sufficient for discoverability: a channel the account has
51 never received a message from has no row to flag. The `/channels` command
52 (step 2 below) therefore upserts candidate rows from `client.get_dialogs()`
53 so users can browse and watch channels they have not yet received traffic
54 from. The flag default and the enumeration flow are orthogonal ? both are
55 required.
56
57 ### 2. Channel enumeration and management commands
58
59 New Saved Messages commands (and, via ADR 0008, bot commands) building on the
60 existing `commands.py` pattern:
61
62 - `/channels` ? list the user's channels (via `client.get_dialogs()`), already
63   used in `backfill.py`) with their watched state.
64 - `/watch <#name|id>` ? set `watched=1` for a source channel.
65 - `/unwatch <#name|id>` ? set `watched=0`.
66 - `/watchlist` ? list only watched sources.
67
68 ### 3. Per-source ? per-destination routing rules
69
70 Introduce a `routing_rules` table (the canonical name; issue #25's `routes`
71 refers to the same concept and should be aligned to `routing_rules`), e.g.:
72
73 ```
74 routing_rules (
75     user_id          INTEGER NOT NULL REFERENCES users(id),
76     source_channel_id INTEGER NOT NULL,
77     dest_channel_id   INTEGER NOT NULL,    -- a broadcast channel / @username
78     filter_config_id  INTEGER,             -- optional per-route filter override
79     PRIMARY KEY (user_id, source_channel_id, dest_channel_id)
80 )
81 ```
82
83 On a match, look up applicable routing rules by `source_channel_id` and
84 deliver to each matching destination. The delivery layer (`delivery.py`)
85 already accepts a `target` per call, so this generalises the existing single
86 `delivery_target` to a per-match lookup. Users with no routing rules fall
87 back to the current global `delivery_target` behaviour (ADR 0005), preserving
88 backwards compatibility.
89
90 ## Consequences
91
92 **Positive**
93 - Enables the multiplexing product shape: scan curated sources, publish to
94   topical broadcast channels, let subscribers pick outputs by interest.
95 - Each capability is additive and independently deployable; none disturbs the
96   existing FORWARD/LINK/BOTH paths or the metadata-first ingestion (ADR 0003).
97 - Dedup (ADR 0006) still holds: Tier 1 (`document_id`) keys the `*original*`
98   file, so a routed re-upload is not re-processed.
99 - Channel enumeration reuses `client.get_dialogs()` already present in
100   `backfill.py`; no new Telethon capability is required.
101
102 **Negative**
103 - New schema (`channels.watched`, `routing_rules`) with migrations.
104 - **Routing loop invariant.** With multiplexed routing, the userbot account
105   may be a member of a `*destination*` broadcast channel and re-see its own
106   output, producing an infinite forward loop. The contract is: **a channel
107   is either a source or a destination, never both.** This is enforced in

```

```

108     `on_new_message` by excluding any channel that is a configured destination
109     (`source_channel_id IN (SELECT dest_channel_id FROM routing_rules WHERE
110     user_id = ?)`) before the watched-flag check. Issue #25 Track E implements
111     this; the invariant is stated here so the handler has a contract to code
112     against.
113 - Per-route filter overrides add config complexity; v1 may ship with global
114   filters only and a single implicit route (`source ? global target`), then
115   layer explicit per-route rules later.
116
117 ## Backwards compatibility
118
119 Adding `channels.watched` with `DEFAULT 0` would silently stop every existing
120 install from receiving matches until the user re-watches their channels ? a
121 behaviour change disguised as a schema change. To preserve the current
122 "ingest everything" semantics for existing installs, the migration backfills:
123
124 ```sql
125 ALTER TABLE channels ADD COLUMN watched INTEGER NOT NULL DEFAULT 0;
126 UPDATE channels SET watched = 1 WHERE seen_first_at IS NOT NULL;
127 ```
128
129 i.e. any channel the install has *already* seen traffic from is implicitly
130 watched, so pre-migration users observe no change. New channels discovered
131 after the migration default to `watched = 0` and must be explicitly watched ?
132 which is the intended new UX. This mirrors the "old rows behave exactly as
133 before" pattern established by ADR 0005.
134
135 ## Rollout order
136
137 1. **Selective scanning** (`channels.watched` + `/watch`, `/unwatch`,
138   `/channels`). Smallest unit; immediately useful even without routing.
139 2. **Named destination channels** ? extend `/set delivery_target` to allow
140   switching among a user-defined set, not just one global value.
141 3. **Per-source ? per-destination routing rules** (`routing_rules` table +
142   `/route <source> <dest>` command). The full multiplexing vision.
143
144 Each step is shippable and useful on its own; step 3 is not required to
145 deliver value from steps 1?2.
146
147 ## Alternatives considered
148
149 - **Server-side `chats=` filter at handler registration:** rejected ? the
150   watched set is per-user and mutable at runtime; re-registering Telethon
151   handlers on every `/watch` change is brittle. An in-handler lookup against
152   the `channels` table is simpler and hot-swappable.
153 - **Content-based classification (auto-detect genre):** explicitly out of
154   scope for v1. Routing is user-defined by source channel; automatic topic
155   detection could be a future enhancement but is not required to deliver the
156   multiplexing UX.
157
158 ## References
159
160 - ADR 0001 (userbot ingestion) ? the capability this selectively constrains.
161 - ADR 0005 (per-user delivery target) ? the single-target delivery this
162   generalises into per-route targets.
163 - ADR 0006 (content dedup) ? still applies; routed re-uploads are keyed by
164   the original `document_id`.
165 - ADR 0008 (bot API onboarding) ? the front-door that exposes `/watch`,
166   `/unwatch`, and routing commands to public users.
167

```

4. user (2026-07-06T09:07:20.585Z)

File does not exist. Note: your current working directory is C:\Users\avidu\Projects\Telegram.

5. assistant (2026-07-06T09:07:23.670Z)

The ADR line references in the prompt are shifted from what I see (the prompt's line numbers differ slightly), but the content is clear. Let me find the actual telethon_client.py and the changed files.

6. user (2026-07-06T09:07:24.256Z)

```
src\tg_video_filter\telethon_client.py
```

7. user (2026-07-06T09:07:26.360Z)

```
db87cc0 feat(scanning): selective source scanning ? /watch, /unwatch, /channels, /watchlist (ADR 0009, Track E)
35f4f70 feat(commands): extract CommandService ? provider-independent command execution (ADR 0011, Track D) (#38)
12d21b9 feat(delivery): destinations, routing_rules, delivery_attempts (ADR 0011, Track C) (#37)
---FILES---
src/tg_video_filter/command_service.py
src/tg_video_filter/commands.py
src/tg_video_filter/db.py
src/tg_video_filter/telethon_client.py
tests/test_command_service.py
```

8. user (2026-07-06T09:07:29.974Z)

```
commit db87cc02915f6343744a6c5c99f84aafd8418da5
Author: Avi Dullu <avi.dullu@gmail.com>
Date: Mon Jul 6 14:31:25 2026 +0530
```

```
feat(scanning): selective source scanning ? /watch, /unwatch, /channels, /watchlist (ADR 0009, Track E)
```

```
diff --git a/src/tg_video_filter/db.py b/src/tg_video_filter/db.py
index bfaadec..bee716b 100644
--- a/src/tg_video_filter/db.py
+++ b/src/tg_video_filter/db.py
@@ -1534,6 +1534,32 @@ async def get_source_by_id(conn: aiomysqlite.Connection, source_id: int) ->
Source
    return _row_to_source(row) if row else None

+async def set_source_watched(
+    conn: aiomysqlite.Connection, account_id: int, source_id: int, watched: bool
+) -> None:
+    """Toggle the ``watched`` flag on a source (ADR 0009, Track E)."""
+    await conn.execute(
+        "UPDATE sources SET watched = ? WHERE id = ? AND account_id = ?",
+        (int(watched), source_id, account_id),
+    )
+    await conn.commit()
+
+async def list_account_sources(
+    conn: aiomysqlite.Connection, account_id: int, *, watched_only: bool = False
+) -> list[Source]:
+    """List all sources for an account, optionally filtering to watched only."""
+    conn.row_factory = aiomysqlite.Row
+    query = "SELECT * FROM sources WHERE account_id = ?"
+    params: tuple = (account_id,)
+    if watched_only:
+        query += " AND watched = 1"
+    query += " ORDER BY watched DESC, provider_source_id ASC"
+    async with conn.execute(query, params) as cur:
```

```

+         rows = await cur.fetchall()
+     return [_row_to_source(r) for r in rows]
+
+
+ async def insert_video(conn: aiosqlite.Connection, video: VideoRecord) -> VideoRecord:
+     """Deprecated shim: insert into ``content_items`` (ADR 0011).

@@ -1717,6 +1743,8 @@ __all__ = [
    "upsert_source",
    "get_source_by_provider_id",
    "get_source_by_id",
+   "set_source_watched",
+   "list_account_sources",
    "insert_content_item",
    "get_content_item_by_message",
    "get_content_item_by_dedup_key",
diff --git a/src/tg_video_filter/telethon_client.py b/src/tg_video_filter/telethon_client.py
index 883e3f8..ebb7416 100644
--- a/src/tg_video_filter/telethon_client.py
+++ b/src/tg_video_filter/telethon_client.py
@@ -181,6 +181,15 @@ def make_new_message_handler(
    message_id = message.id
    document_id = _extract_document_id(message)

+   # --- watched gate (ADR 0009, Track E) -----
+   # Skip ingestion for unwatched sources. The channel must already be in
+   # sources (lazy-created by the /channels enumeration or the legacy
+   # upsert_channel below); if absent, we let the legacy path upsert it
+   # with watched=True for backwards-compatibility.
+   src = await db.get_source_by_provider_id(conn, user.id, "telegram", str(channel_id))
+   if src is not None and not src.watched:
+       return # silently skip unwatched source
+
+   # --- three-tier dedup (cheapest first) ---
+
+   # Tier 0: same (user, channel, message) instance ? already processed.

```

9. assistant (2026-07-06T09:07:32.641Z)

Now let me see the surrounding context of the gate ? the ordering relative to upsert_channel and delivery ? and the upsert_channel / upsert_source watched default.

10. user (2026-07-06T09:07:33.637Z)

```

160         plus the existing (channel, message) instance dedup,
161         3. runs the filter engine (read from ``engine_holder`` so config
162            changes via ``/set`` take effect immediately),
163         4. inserts a VideoRecord row,
164         5. delivers a match to ``delivery_target`` (see ADR 0005).
165     """
166
167     async def on_new_message(event: NewMessage.Event) -> None: # pragma: no cover
168         message: Message = event.message # type: ignore[assignment]
169         if message is None:
170             return # Telethon edge case: event has no message (shouldn't happen)
171         # Only process genuine channel/group posts and private messages we
172         # receive ? outgoing messages (sent by our own account) are ignored.
173         if getattr(message, "out", False):
174             return
175
176         meta = extract_video_meta(message)
177         if meta is None:
178             return # not a video
179
180         channel_id = _peer_id(message.peer_id)

```

```

181         message_id = message.id
182         document_id = _extract_document_id(message)
183
184         # --- watched gate (ADR 0009, Track E) -----
185         # Skip ingestion for unwatched sources. The channel must already be in
186         # sources (lazy-created by the /channels enumeration or the legacy
187         # upsert_channel below); if absent, we let the legacy path upsert it
188         # with watched=True for backwards-compatibility.
189         src = await db.get_source_by_provider_id(conn, user.id, "telegram",
190 str(channel_id))
191         if src is not None and not src.watched:
192             return # silently skip unwatched source
193
194         # --- three-tier dedup (cheapest first) ---
195
196         # Tier 0: same (user, channel, message) instance ? already processed.
197         existing = await db.get_video(conn, user.id, channel_id, message_id)
198         if existing is not None:
199             return
200
201         # Tier 1: same Telegram document.id ? forwarded/cross-posted file.
202         if document_id is not None:
203             existing = await db.get_video_by_document_id(conn, user.id, document_id)
204             if existing is not None:
205                 log.info(
206                     "video_dedup_document_id user=%s doc_id=%s channel=%s msg=%s "
207                     "original_channel=%s original_msg=%s",
208                     user.id,
209                     document_id,
210                     channel_id,
211                     message_id,
212                     existing.channel_id,
213                     existing.message_id,
214                 )
215                 return
216
217         # Tier 2: metadata fingerprint ? same file re-uploaded (rare but possible).
218         if (
219             meta.size_bytes is not None
220             and meta.duration_s is not None
221             and meta.height is not None
222             and meta.width is not None
223         ):
224             existing = await db.get_video_by_content_fingerprint(
225                 conn,
226                 user.id,
227                 meta.size_bytes,
228                 meta.duration_s,
229                 meta.height,
230                 meta.width,
231             )
232             if existing is not None:
233                 log.info(
234                     "video_dedup_fingerprint user=%s size=%s dur=%s h=%s w=%s "
235                     "channel=%s msg=%s original_channel=%s original_msg=%s",
236                     user.id,
237                     meta.size_bytes,
238                     meta.duration_s,
239                     meta.height,
240                     meta.width,
241                     channel_id,
242                     message_id,
243                     existing.channel_id,
244                     existing.message_id,
245                 )

```

```

245         return
246
247     # Lazily record the channel (best-effort; failures are non-fatal).
248     try:
249         await db.upsert_channel(
250             conn,
251             Channel(user_id=user.id, channel_id=channel_id,
252 title=_channel_title(event)),
253         )
254     except Exception:
255         log.warning(
256             "failed to upsert channel %s for user %s", channel_id, user.id,
257 exc_info=True
258         )
259
260     result = engine_holder[ENGINE_KEY].evaluate(meta)
261     record = VideoRecord(
262         user_id=user.id,
263         channel_id=channel_id,
264         message_id=message_id,
265         document_id=document_id,
266         duration_s=meta.duration_s,
267         width=meta.width,
268         height=meta.height,
269         size_bytes=meta.size_bytes,
270         mime_type=meta.mime_type,
271         matched=result.matched,
272     )
273     try:
274         inserted = await db.insert_video(conn, record)
275     except sqlite3.IntegrityError:
276         # Lost a dedup race: a concurrent delivery of the same file
277         # (same document_id or same message instance) won the insert.
278         # The three-tier check above already classified this as new, so
279         # the collision means another task is processing (or has
280         # processed) it ? log and skip.
281         log.info(

```

11. assistant (2026-07-06T09:07:37.344Z)

The seed is confirmed by reading. The gate at line 189-191 only returns early when `src` is not None and not `src.watched`. A brand-new channel has no source row (`src` is None), so it falls through and gets processed, delivered, then `upsert\_channel` lazily records it. Let me confirm `upsert\_channel`'s watched default and check `upsert\_source` too.

12. user (2026-07-06T09:07:38.936Z)

```

99:     watched          INTEGER NOT NULL DEFAULT 0,
522:     # 1a. Backfill sources from channels first (known title + watched=1). Order
523:     #   matters: channels-first so title/watched data wins; the videos-based
528:     #   (account_id, provider, provider_source_id, title, kind, watched,
539:     #   ? the channels-based row above survives with its title + watched=1.
543:     #   (account_id, provider, provider_source_id, kind, watched, seen_first_at)
851:     watched=bool(row["watched"]),
1104:async def upsert_source(conn: aiosqlite.Connection, source: Source) -> Source:
1105:     """Insert a source row, or update title/watched on conflict.
1112:     #   (account_id, provider, provider_source_id, title, kind, watched,
1117:     #   watched=excluded.watched,
1125:     #   int(source.watched),
1387:async def upsert_channel(conn: aiosqlite.Connection, channel: Channel) -> Channel:
1402:     #   watched=True, # back-compat: pre-ADR-0011 channels were all watched
1537:async def set_source_watched(
1538:     conn: aiosqlite.Connection, account_id: int, source_id: int, watched: bool
1540:     """Toggle the ``watched`` flag on a source (ADR 0009, Track E)."""
1542:     "UPDATE sources SET watched = ? WHERE id = ? AND account_id = ?",

```

```

1543:         (int(watched), source_id, account_id),
1549:     conn: aiosqlite.Connection, account_id: int, *, watched_only: bool = False
1551:     """List all sources for an account, optionally filtering to watched only."""
1555:     if watched_only:
1556:         query += " AND watched = 1"
1557:     query += " ORDER BY watched DESC, provider_source_id ASC"
1578:         watched=True,
1746:     "set_source_watched",

```

13. user (2026-07-06T09:07:45.977Z)

```

1387     async def upsert_channel(conn: aiosqlite.Connection, channel: Channel) -> Channel:
1388         """Deprecated shim: upsert the :class:`Source` implied by a legacy Channel.
1389
1390         ADR 0011: ``channels`` is renamed to ``channels_deprecated``; this function
1391         reads/writes ``sources`` instead and maps the legacy ``Channel`` shape back.
1392         New code should call :func:`upsert_source` directly.
1393         """
1394         src = await upsert_source(
1395             conn,
1396             Source(
1397                 account_id=channel.user_id,
1398                 provider="telegram",
1399                 provider_source_id=str(channel.channel_id),
1400                 title=channel.title,
1401                 kind="channel",
1402                 watched=True, # back-compat: pre-ADR-0011 channels were all watched
1403                 seen_first_at=channel.seen_first_at,
1404             ),
1405         )
1406         return _source_to_channel(src)
1407
1408
1409     async def get_channel(conn: aiosqlite.Connection, user_id: int, channel_id: int) ->
Channel | None:
1410         """Deprecated shim: project the :class:`Source` row back to a legacy Channel."""
1411         src = await get_source_by_provider_id(conn, user_id, "telegram", str(channel_id))
1412         return _source_to_channel(src) if src else None
1413
1414
1415     def _source_to_channel(src: Source) -> Channel:
1416         """Project a provider-neutral :class:`Source` onto the legacy :class:`Channel`.
```

14. assistant (2026-07-06T09:07:52.360Z)

Confirmed. `upsert\_channel` at line 1402 hardcodes `watched=True`. This is the core defect. Now let me verify the reproduction empirically and check the `\_resolve\_source` and `/channels` upsert behavior in command_service.py to assess the discoverability requirement (ADR line 48-54: `/channels` should upsert candidates from get_dialogs so unseen channels get rows at watched=0).

15. user (2026-07-06T09:07:52.636Z)

```

1     """Provider-independent command parsing and execution (ADR 0011, Track D).
2
3     Extracts command logic from the Telethon adapter (`commands.py`) so the
4     same ``/set``, ``/show`` commands work identically through a Bot API
5     front-door. The service is pure: strings in, ``CommandResult`` out ? no
6     Telethon objects, no network calls.
7
8     See ``docs/adr/0007-inline-config-commands.md`` for the command model and
9     ``docs/adr/0008-bot-api-onboarding-front-door.md`` for the Bot API motivation.
10
11
12     from __future__ import annotations

```

```

13
14 import logging
15 import re
16 from dataclasses import dataclass
17
18 import aiosqlite
19
20 from tg_video_filter import db
21 from tg_video_filter.models import FilterConfig, Source
22
23 log = logging.getLogger(__name__)
24
25 # Key into the shared engine_holder dict (imported by commands.py's adapter).
26 ENGINE_KEY = "engine"
27
28 # --- regex patterns -----
29
30 _SIZE_RE = re.compile(r"^/set\s+size\s+(\d+)\s*([kmgT]?b?)$", re.IGNORECASE)
31 _LENGTH_RE = re.compile(r"^/set\s+(?:max\s+)?length\s+(\d+)\s*([smh]?)", re.IGNORECASE)
32 _MIN_LENGTH_RE = re.compile(r"^/set\s+min\s+length\s+(\d+)\s*([smh]?)", re.IGNORECASE)
33 _MIN_RES_RE = re.compile(r"^/set\s+min\s+(?:resolution|res)\s+(\d+)\s*p?$",
re.IGNORECASE)
34 _SHOW_CONFIG_RE = re.compile(r"^/show\s+config$", re.IGNORECASE)
35 _SHOW_COMMANDS_RE = re.compile(r"^/show\s+commands$", re.IGNORECASE)
36 _EXCLUDE_RE = re.compile(r"^/set\s+exclude\s+(gif|round|nosound)\s+(on|off)$",
re.IGNORECASE)
37 _EXCLUDE_HELP_RE = re.compile(r"^/set\s+exclude\s+", re.IGNORECASE)
38 _MIN_RES_HELP_RE = re.compile(r"^/set\s+min\s+(?:resolution|res)\s+", re.IGNORECASE)
39
40
41 # --- source scanning regex (ADR 0009, Track E) -----
42
43 _WATCH_RE = re.compile(r"^/watch\s+(.+)$", re.IGNORECASE)
44 _UNWATCH_RE = re.compile(r"^/unwatch\s+(.+)$", re.IGNORECASE)
45 _CHANNELS_RE = re.compile(r"^/channels$", re.IGNORECASE)
46 _WATCHLIST_RE = re.compile(r"^/watchlist$", re.IGNORECASE)
47
48 # --- unit parsing -----
49
50
51 def parse_size_mb(value_str: str, unit: str) -> int:
52     """Parse a size value + unit string into mebibytes."""
53     value = int(value_str)
54     unit = unit.lower().rstrip("b")
55     multipliers: dict[str, int] = {"": 1, "m": 1, "k": 1, "g": 1024, "t": 1024 * 1024}
56     if unit in multipliers:
57         raw = value * multipliers[unit]
58         if unit == "k":
59             raw = max(1, raw // 1024)
60         return raw
61     raise ValueError(f"Unknown size unit: {unit}")
62
63
64 def parse_duration_s(value_str: str, unit: str) -> int:
65     """Parse a duration value + unit string into seconds."""
66     value = int(value_str)
67     unit = unit.lower()
68     multipliers: dict[str, int] = {"": 1, "s": 1, "m": 60, "h": 3600}
69     if unit in multipliers:
70         return value * multipliers[unit]
71     raise ValueError(f"Unknown time unit: {unit}")
72
73
74 # --- formatting helpers -----
75

```

```

76
77 def _format_duration(seconds: int | None) -> str:
78     if seconds is None:
79         return "unlimited"
80     if seconds == 0:
81         return "0s"
82     if seconds < 60:
83         return f"{seconds}s"
84     if seconds < 3600:
85         m, s = divmod(seconds, 60)
86         return f"{m}m{s}s" if s else f"{m}m"
87     h, rem = divmod(seconds, 3600)
88     m, s = divmod(rem, 60)
89     parts = f"{h}h"
90     if m:
91         parts += f"{m}m"
92     if s:
93         parts += f"{s}s"
94     return parts
95
96
97 def _format_size(mb: int | None) -> str:
98     if mb is None:
99         return "unlimited"
100     if mb == 0:
101         return "0MB"
102     if mb >= 1024:
103         gb = mb / 1024
104         if gb == int(gb):
105             return f"{int(gb)}GB"
106         return f"{gb:.1f}GB"
107     return f"{mb}MB"
108
109
110 def format_config(cfg: FilterConfig) -> str:
111     """Format a ``FilterConfig`` as a readable message (Telegram markdown)."""
112     lines = [
113         "? **Current Filter Configuration**",
114         "",
115         f"?? **Duration:** {_format_duration(cfg.min_duration_s)} - "
116         f"{_format_duration(cfg.max_duration_s)}",
117         f"? **Min resolution:** {'?' + str(cfg.target_height) + 'p' if "
118         f"cfg.target_height else 'disabled'}",
119         f"? **Max file size:** {_format_size(cfg.max_size_mb)}",
120         f"? **Delivery mode:** `{cfg.delivery_mode.value}`",
121         f"? **Delivery target:** `{cfg.delivery_target}`",
122         "",
123         "? **Content exclusions:**",
124         f"    GIF/animated: {'?' excluded' if cfg.exclude_gif else '? allowed'}",
125         f"    Round videos: {'?' excluded' if cfg.exclude_round else '? allowed'}",
126         f"    Silent (no audio): {'?' excluded' if cfg.exclude_nosound else '? allowed'}",
127     ]
128     return "\n".join(lines)
129
130 def format_commands() -> str:
131     """Return a help message listing all available commands."""
132     return "\n".join(
133         [
134             "? **Available Commands**",
135             "",
136             "? **Filter config**",
137             "`/show config` ? Show current filter settings",
138             "`/set size <N><unit>` ? Set max file size (e.g. `/set size 10MB`)",
139             "    Units: `MB` (default), `GB`, `KB` ? case-insensitive, optional trailing

```

```

`B`",
139         "/set length <N><unit>` ? Set max video duration (e.g. `/set length 15m`)",
140         "    or `/set max length <N><unit>` ? same as above",
141         "    Units: `s` (default), `m`, `h` ? case-insensitive",
142         "/set min length <N><unit>` ? Set min video duration (e.g. `/set min length
5s`)",
143         "    Units: `s` (default), `m`, `h` ? case-insensitive",
144         "/set min resolution <N>p` ? Set min video height (e.g. `/set min
resolution 720p`)",
145         "    or `/set min res <N>p` ? same as above",
146         "",
147         "? **Content exclusions** (all off by default)",
148         "/set exclude gif on|off` ? Exclude Telegram GIFs/animated documents",
149         "/set exclude round on|off` ? Exclude circular video messages",
150         "/set exclude nosound on|off` ? Exclude videos without audio\n"
151         "    Note: GIFs are usually silent; `exclude nosound` will catch most GIFs
too.",
152         "",
153         "? **Backfill**",
154         "/backfill [N][hdw]` ? Scan last N hours/days/weeks of channel history
(default 90d)",
155         "/stop` ? Cancel running backfill",
156         "",
157         "\u2139\u2013 **Help**",
158         "/show commands` ? Show this help message",
159     ]
160 )
161
162
163 # --- command result -----
164
165
166 @dataclass
167 class CommandResult:
168     """The output of executing a command ? provider-independent.
169
170     The adapter (Telethon or Bot API) uses ``reply_text`` to send the response
171     and ``config_changed`` to decide whether to hot-reload the filter engine.
172     """
173
174     reply_text: str = ""
175     config_changed: bool = False
176
177
178 # --- command service -----
179
180
181 class CommandService:
182     """Provider-independent command execution.
183
184     Parses a command string against an account's current config and DB,
185     returning a ``CommandResult``. No Telethon objects involved ? pure string
186     in, effects out. The Telethon adapter (``commands.py``) handles the
187     transport (checking message.out, sending the reply); the Bot API adapter
188     (future) calls ``handle()`` with the ``from.id`` from the update.
189
190     Usage::
191
192         svc = CommandService()
193         result = await svc.handle(user_id, text, conn, current_cfg)
194         if result.config_changed:
195             engine_holder["engine"] = FilterEngine(current_cfg)
196         await send_reply(result.reply_text)
197     """
198

```

```

199 # -- public API -----
200
201 async def handle(
202     self,
203     user_id: int,
204     text: str,
205     conn: aiosqlite.Connection,
206     current_cfg: FilterConfig,
207 ) -> CommandResult:
208     """Parse and execute a command text, returning the reply + change flag."""
209     text = text.rstrip()
210
211     # --- /set size <value><unit> -----
212     m = _SIZE_RE.match(text)
213     if m:
214         value_str, unit = m.group(1), m.group(2) or ""
215         # NOTE: ValueError is unreachable here ? the regex already validates
216         # the unit against [kmgT]?b?, so parse_size_mb can never raise.
217         # Kept for defensive parity with the original commands.py refactor.
218         try:
219             new_mb = parse_size_mb(value_str, unit)
220         except ValueError:
221             return CommandResult("? Invalid size format. Use e.g. `/set size
10MB`.")
222         if new_mb < 1:
223             return CommandResult("? Size must be at least 1 MB (`FilterConfig`
constraint).")
224         cfg = await self._update(conn, user_id, current_cfg, max_size_mb=new_mb)
225         return CommandResult(
226             f"? Max file size set to
**[_format_size(new_mb)]**.\n\n{format_config(cfg)}",
227             config_changed=True,
228         )
229
230     # --- /set length <value><unit> -----
231     m = _LENGTH_RE.match(text)
232     if m:
233         value_str, unit = m.group(1), m.group(2) or ""
234         try:
235             new_secs = parse_duration_s(value_str, unit)
236         except ValueError:
237             return CommandResult("? Invalid duration format. Use e.g. `/set length
15m`.")
238         min_secs = current_cfg.min_duration_s or 0
239         if new_secs < min_secs:
240             return CommandResult(
241                 f"? Max duration ({_format_duration(new_secs)}) must be "
242                 f">= min duration ({_format_duration(min_secs)})."
243             )
244         cfg = await self._update(conn, user_id, current_cfg,
max_duration_s=new_secs)
245         return CommandResult(
246             f"? Max duration set to
**[_format_duration(new_secs)]**.\n\n{format_config(cfg)}",
247             config_changed=True,
248         )
249
250     # --- /set min length <value><unit> -----
251     m = _MIN_LENGTH_RE.match(text)
252     if m:
253         value_str, unit = m.group(1), m.group(2) or ""
254         try:
255             new_min = parse_duration_s(value_str, unit)
256         except ValueError:
257             return CommandResult("? Invalid duration format. Use e.g. `/set min

```

```

length 5s`.")
258         max_secs = current_cfg.max_duration_s or 0
259         if max_secs > 0 and new_min > max_secs:
260             return CommandResult(
261                 f"? Min duration ({_format_duration(new_min)}) must be "
262                 f"<= max duration ({_format_duration(max_secs)})."
263             )
264         cfg = await self._update(conn, user_id, current_cfg, min_duration_s=new_min)
265         return CommandResult(
266             f"? Min duration set to
**{_format_duration(new_min)}**.\n\n{format_config(cfg)}",
267             config_changed=True,
268         )
269
270     # --- /set min resolution <N>p -----
271     m = _MIN_RES_RE.match(text)
272     if m:
273         new_height = int(m.group(1))
274         if new_height < 1:
275             return CommandResult("? Resolution must be at least 1p.")
276         cfg = await self._update(conn, user_id, current_cfg,
target_height=new_height)
277         return CommandResult(
278             f"? Min resolution set to **{new_height}p**.\n\n{format_config(cfg)}",
279             config_changed=True,
280         )
281
282     if _MIN_RES_HELP_RE.match(text):
283         return CommandResult(
284             "? Usage: `/set min resolution <N>p` (e.g. `/set min resolution 720p`)"
285         )
286
287     # --- /set exclude <gif|round|nosound> <on|off> -----
288     m = _EXCLUDE_RE.match(text)
289     if m:
290         what = m.group(1).lower()
291         state = m.group(2).lower()
292         enabled = state == "on"
293         labels = {"gif": "GIF/animated", "round": "round videos", "nosound": "silent
videos"}
294         field_map = {
295             "gif": "exclude_gif",
296             "round": "exclude_round",
297             "nosound": "exclude_nosound",
298         }
299         cfg = await self._update(conn, user_id, current_cfg, **{field_map[what]:
enabled})
300         return CommandResult(
301             f"? {labels[what]} exclusion: **{'ON' if enabled else 'OFF'}**."
302             f"\n\n{format_config(cfg)}",
303             config_changed=True,
304         )
305
306     if _EXCLUDE_HELP_RE.match(text):
307         return CommandResult(
308             "? Usage: `/set exclude <gif|round|nosound> <on|off>`\n"
309             "Example: `/set exclude gif on`"
310         )
311
312     # --- /show config -----
313     if _SHOW_CONFIG_RE.match(text):
314         return CommandResult(format_config(current_cfg))
315
316     # --- source scanning commands (ADR 0009, Track E) -----
317

```

```

318         m = _WATCH_RE.match(text)
319         if m:
320             return await self._handle_watch(conn, user_id, m.group(1).strip(),
current_cfg)
321
322         m = _UNWATCH_RE.match(text)
323         if m:
324             return await self._handle_unwatch(conn, user_id, m.group(1).strip(),
current_cfg)
325
326         if _CHANNELS_RE.match(text):
327             return await self._handle_channels(conn, user_id)
328
329         if _WATCHLIST_RE.match(text):
330             return await self._handle_watchlist(conn, user_id)
331
332         # --- /show commands -----
333         if _SHOW_COMMANDS_RE.match(text):
334             return CommandResult(format_commands())
335
336         # Not a recognised command ? return empty result (noise).
337         return CommandResult()
338
339     # -- source scanning handlers (ADR 0009, Track E) -----
340
341     async def _handle_watch(
342         self, conn:aiosqlite.Connection, user_id:int, target:str, _cfg:FilterConfig
343     ) -> CommandResult:
344         """Parse ``/watch <@username|channel_id>`` and toggle ``watched=1``."""
345         src = await self._resolve_source(conn, user_id, target)
346         if src is None:
347             return CommandResult(
348                 f"? Source `{target}` not found. Use `/channels` to list "
349                 f"available sources.\n\nSources are discovered as messages "
350                 f"arrive; a channel you haven't received traffic from may not "
351                 f"appear yet."
352             )
353         await db.set_source_watched(conn, user_id, src.id, True) # type: ignore[arg-
type]
354         label = src.title or src.provider_source_id
355         return CommandResult(f"? Now watching **{label}**
({src.provider_source_id}).")
356
357     async def _handle_unwatch(
358         self, conn:aiosqlite.Connection, user_id:int, target:str, _cfg:FilterConfig
359     ) -> CommandResult:
360         """Parse ``/unwatch <@username|channel_id>`` and toggle ``watched=0``."""
361         src = await self._resolve_source(conn, user_id, target)
362         if src is None:
363             return CommandResult(
364                 f"? Source `{target}` not found. Use `/channels` to list sources."
365             )
366         await db.set_source_watched(conn, user_id, src.id, False) # type: ignore[arg-
type]
367         label = src.title or src.provider_source_id
368         return CommandResult(f"? Stopped watching **{label}**
({src.provider_source_id}).")
369
370     async def _handle_channels(self, conn:aiosqlite.Connection, user_id:int) ->
CommandResult:
371         """List all known sources with watch status."""
372         sources = await db.list_account_sources(conn, user_id)
373         if not sources:
374             return CommandResult(
375                 "? **No sources yet.**\n\n"

```

```

376         "Sources are discovered when a new message arrives from a "
377         "channel. Run `/backfill` to discover historical channels, "
378         "or wait for new traffic."
379     )
380     lines = ["? **All sources**", ""]
381     for s in sources:
382         icon = "?" if s.watched else "?"
383         label = s.title or s.provider_source_id
384         lines.append(f"{icon} `{s.provider_source_id}` ? {label}")
385     return CommandResult("\n".join(lines))
386
387     async def _handle_watchlist(self, conn: aiomysqlite.Connection, user_id: int) ->
CommandResult:
388         """List only watched sources."""
389         sources = await db.list_account_sources(conn, user_id, watched_only=True)
390         if not sources:
391             return CommandResult(
392                 "? **No watched sources.**\n\n"
393                 "Use `/channels` to see available sources, then `/watch <id>` "
394                 "to start scanning."
395             )
396         lines = ["? **Watched sources**", ""]
397         for s in sources:
398             label = s.title or s.provider_source_id
399             lines.append(f"? `{s.provider_source_id}` ? {label}")
400         return CommandResult("\n".join(lines))
401
402     async def _resolve_source(
403         self, conn: aiomysqlite.Connection, user_id: int, target: str
404     ) -> Source | None:
405         """Resolve a user-provided source identifier to a ``Source`` row.
406
407         Accepts ``@username`` or a numeric channel id string. The ``@`` prefix
408         is stripped before lookup since Telegram channel ids are numeric.
409         """
410         target = target.lstrip("@")
411         # Try by provider-native id first (the common case).
412         src = await db.get_source_by_provider_id(conn, user_id, "telegram", target)
413         if src is not None:
414             return src
415         # Fall back: try with @ prefix in the target column (just in case).
416         return await db.get_source_by_provider_id(conn, user_id, "telegram",
f"@{target}")
417
418     # -- internal -----
419
420     async def _update(
421         self,
422         conn: aiomysqlite.Connection,
423         user_id: int,
424         current: FilterConfig,
425         max_size_mb: int | None = None,
426         max_duration_s: int | None = None,
427         min_duration_s: int | None = None,
428         target_height: int | None = None,
429         exclude_gif: bool | None = None,
430         exclude_round: bool | None = None,
431         exclude_nosound: bool | None = None,
432     ) -> FilterConfig:
433         """Apply partial updates to the account's config, persist, and return."""
434         updates: dict[str, object] = {}
435         if max_size_mb is not None:
436             updates["max_size_mb"] = max_size_mb
437         if max_duration_s is not None:
438             updates["max_duration_s"] = max_duration_s

```

```

439         if min_duration_s is not None:
440             updates["min_duration_s"] = min_duration_s
441         if target_height is not None:
442             updates["target_height"] = target_height
443         if exclude_gif is not None:
444             updates["exclude_gif"] = exclude_gif
445         if exclude_round is not None:
446             updates["exclude_round"] = exclude_round
447         if exclude_nosound is not None:
448             updates["exclude_nosound"] = exclude_nosound
449         cfg = current.model_copy(update=updates) if updates else current
450         await db.save_filter_config(conn, user_id, cfg)
451         log.info(
452             "config_updated user=%s max_size_mb=%s min_duration_s=%s max_duration_s=%s "
453             "target_height=%s exclude_gif=%s exclude_round=%s exclude_nosound=%s",
454             user_id,
455             cfg.max_size_mb,
456             cfg.min_duration_s,
457             cfg.max_duration_s,
458             cfg.target_height,
459             cfg.exclude_gif,
460             cfg.exclude_round,
461             cfg.exclude_nosound,
462         )
463         return cfg
464
465
466     __all__ = [
467         "ENGINE_KEY",
468         "CommandResult",
469         "CommandService",
470         "_format_duration",
471         "_format_size",
472         "format_config",
473         "format_commands",
474         "parse_duration_s",
475         "parse_size_mb",
476     ]
477

```

16. assistant (2026-07-06T09:07:57.774Z)

Now I have a full picture. Let me empirically reproduce the core defect to confirm the ordering:
new channel with no source row ? processed + delivered + auto-watched.

17. user (2026-07-06T09:10:06.687Z)

Command running in background with ID: b23182rbj. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\b23182rbj.output. You will be notified when it completes. To check interim output, use Read on that file path.

18. user (2026-07-06T09:12:11.487Z)

Command running in background with ID: bhte6wger. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\bhte6wger.output. You will be notified when it completes. To check interim output, use Read on that file path.

19. user (2026-07-06T09:12:16.813Z)

```

1     Traceback (most recent call last):
2         File "C:\Users\avidu\AppData\Local\Temp\claude\repro.py", line 31, in <module>
3             asyncio.run(main())

```

```

4         ~~~~~^
5         File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py",
line 195, in run
6             return runner.run(main)
7             ~~~~~^
8         File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py",
line 118, in run
9             return self._loop.run_until_complete(task)
10            ~~~~~^
11         File
"C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line 725,
in run_until_complete
12             return future.result()
13             ~~~~~^
14         File "C:\Users\avidu\AppData\Local\Temp\claude\repro.py", line 8, in main
15             acc = await db.upsert_account(conn, Account(provider="telegram",
provider_account_id="u1", label="t"))
16
~~~~~^
17         File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\pydantic\main.py", line
263, in __init__
18             validated_self = self.__pydantic_validator__.validate_python(data,
self_instance=self)
19         pydantic_core._pydantic_core.ValidationError: 1 validation error for Account
20             id
21             Field required [type=missing, input_value={'provider': 'telegram', ...id': 'u1',
'label': 't'}, input_type=dict]
22             For further information visit https://errors.pydantic.dev/2.13/v/missing
23

```

20. assistant (2026-07-06T09:12:18.714Z)

Let me check the Account/Source model signatures.

21. user (2026-07-06T09:12:20.393Z)

```

35:class Account(BaseModel):
123:class Source(BaseModel):

```

22. user (2026-07-06T09:12:24.840Z)

```

35     class Account(BaseModel):
36         """A product identity ? the owner of config, channels, routes, and content.
37
38         Decoupled from any specific provider's session (ADR 0010). ``telegram_id``
39         is the bridge used by the Bot API front-door (``from.id``) and by the
40         legacy userbot path; it is nullable so future providers that don't have a
41         Telegram id (e.g. WhatsApp) can still own an ``Account``.
42         """
43
44         model_config = ConfigDict(frozen=True)
45
46         id: int = Field(description="Stable surrogate PK; the owner key for all downstream
tables")
47         telegram_id: int | None = Field(
48             default=None,
49             description="Telegram user id (bot from.id OR userbot account id). UNIQUE when
set.",
50         )
51         provider: str = Field(default="telegram", description="Provider namespace; future:
'whatsapp'")
52         display_name: str | None = None
53         created_at: datetime = Field(default_factory=_utcnow)
54

```

```

55
56     class TelegramSession(BaseModel):
57         """An optional MTProto (Telethon) session linked to an :class:`Account`.
58
59         A bot-only user has an ``Account`` row but no ``TelegramSession`` row;
60         that is what makes them structurally invisible to ``manager.run``
61         (ADR 0010). ``session_path`` is data-driven so the file-location
62         convention is explicit rather than derived from the PK at runtime.
63         """
64 
```

The Account model requires `id` and uses `telegram\_id`. Let me check upsert_account's actual signature to seed correctly.

```

936:async def upsert_account(conn: aioredis.Connection, account: Account) -> Account:
937-    """Insert or update an account row; returns the persisted Account."""
938-    await conn.execute(
939-        """INSERT INTO accounts (id, telegram_id, provider, display_name, created_at)
940-        VALUES (?, ?, ?, ?, ?)
941-        ON CONFLICT(id) DO UPDATE SET
942-            telegram_id=excluded.telegram_id,
943-            provider=excluded.provider,
944-            display_name=excluded.display_name""",
945-        (
946-            account.id,
947-            account.telegram_id,
948-            account.provider,

```

[illegible]

```
Traceback (most recent call last):
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\threading.py", line 1044, in _bootstrap_inner
    self.run()
    ~~~~~^^
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\threading.py", line 995, in run
    self._target(*self._args, **self._kwargs)
    ~~~~~^~~~~~
```

[illegible]

26. assistant (2026-07-06T09:12:46.905Z)

The core defect is empirically confirmed (the trailing exception is just the connection-close teardown, irrelevant to the result):

- ```
- BEFORE msg: no source row exists
- Gate does NOT skip a brand-new channel ? it gets scanned, deduped, and delivered
- AFTER lazy `upsert_channel`: `watched=True` ? auto-watched (OPT-OUT)
- 2nd message: gate still does not skip ? keeps scanning
```

This directly violates ADR 0009 (opt-in DEFAULT 0; new channels default watched=0 and must be explicitly watched; return early before any dedup/filter work; freshly-onboarded user scans nothing until they opt in).

Now let me assess the second ADR requirement gap: ``/channels`` should upsert candidates from ``get_dialogs()`` (discoverability). The ``_handle_channels`` at line 370-385 only lists existing sources; it never calls ``get_dialogs()``. And ``_resolve_source`` only resolves numeric IDs / ``@``-prefixed native ids, not channel name/username as ADR line 64 (``/watch <#name|id>``) implies. Let me verify what ``provider_source_id`` holds and whether username resolution is possible.

## 27. user (2026-07-06T09:12:47.650Z)

```

123 class Source(BaseModel):
124 """A configured place we ingest content from (ADR 0011).
125
126 Generalizes the legacy :class:`Channel` (1:1 for Telegram today; future
127 providers add their own source kinds). ``provider_source_id`` is the
128 provider-native id (Telegram ``channel_id``, future WhatsApp group id, ?).
129 """
130
131 model_config = ConfigDict(frozen=True)
132
133 id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
134 account_id: int = Field(description="FK ? accounts.id (owner of the source)")
135 provider: str = Field(default="telegram", description="Provider namespace")
136 provider_source_id: str = Field(
137 description="Provider-native source id (Telegram channel_id, ?)"
138)
139 title: str | None = None
140 kind: str = Field(default="channel", description="'channel' | 'group' | 'feed'")
141 watched: bool = Field(
142 default=False,
143 description="ADR 0009 selective scanning: ingest from this source iff True",
144)
145 seen_first_at: datetime = Field(default_factory=utcnow)
146 last_seen_at: datetime | None = None
147
148
149 class ContentItem(BaseModel):
150 """A normalized content record discovered from any :class:`Source` (ADR 0011).
151
152 Generalizes the legacy :class:`VideoRecord`. Phase 1 stores only videos

```

## 28. assistant (2026-07-06T09:12:54.053Z)

Now I have everything needed. Let me verify the `commands.py` re-export change too, to confirm no separate defect there, and check the two remaining items:

1. `\_resolve\_source` only matches by numeric `provider\_source\_id` ? ADR line 64 says `/watch <#name|id>` (by name/username OR id). `provider\_source\_id` stores the numeric channel\_id. So watching by channel name/username does not work.
2. `/channels` doesn't upsert from `get\_dialogs()`.

Let me check the commands.py diff briefly.

## 29. user (2026-07-06T09:12:55.505Z)

commit db87cc02915f6343744a6c5c99f84aafd8418da5

Author: Avi Dullu <avi.dullu@gmail.com>

Date: Mon Jul 6 14:31:25 2026 +0530

feat(scanning): selective source scanning ? /watch, /unwatch, /channels, /watchlist (ADR 0009, Track E)

```
diff --git a/src/tg_video_filter/commands.py b/src/tg_video_filter/commands.py
index 974be99..3aea1f6 100644
```

```
--- a/src/tg_video_filter/commands.py
```

```
+++ b/src/tg_video_filter/commands.py
```

```
@@ -27,7 +27,8 @@ from tg_video_filter.models import FilterConfig, User
```

```
log = logging.getLogger(__name__)
```

```
-from tg_video_filter.command_service import (# noqa: E402
```

```
+from tg_video_filter.command_service import (# noqa: E402, F401
```

```
+ _CHANNELS_RE,
+ _EXCLUDE_HELP_RE,
+ _EXCLUDE_RE,
+ _LENGTH_RE,
```

```
@@ -37,6 +38,9 @@ from tg_video_filter.command_service import (# noqa: E402
```

```
 _SHOW_COMMANDS_RE,
 _SHOW_CONFIG_RE,
 _SIZE_RE,
```

```
+ _UNWATCH_RE,
```

```
+ _WATCH_RE,
```

```
+ _WATCHLIST_RE,
+ _format_duration,
+ _format_size,
+ format_commands,
```

## 30. assistant (2026-07-06T09:12:59.956Z)

commands.py just re-exports the regex patterns for backward compat ? benign.

Now, one more thing to verify for the discoverability finding ? does `/backfill` populate sources with `watched=1` or `0`? The `\_handle\_channels` empty-state message tells the user to run `/backfill` to discover channels. If backfill upserts sources (via the same lazy `upsert\_channel` path) with `watched=True`, that compounds the opt-out problem. Let me check.

## 31. user (2026-07-06T09:13:01.593Z)

```
120: dialogs = await client.get_dialogs()
```

## 32. assistant (2026-07-06T09:13:09.650Z)

backfill.py wasn't changed by this PR, so I'll keep my findings scoped to the PR diff. The key point: `\_handle\_channels` (new code) never calls `get\_dialogs()` to upsert candidate rows, contrary to ADR lines 48-54, and its empty-state text points to `/backfill` instead. That's a

discoverability deviation in the new code.

Let me finalize. I have three findings ranked by severity:

1. **\*\*Blocker/High\*\***: The ingestion gate auto-watches brand-new channels (opt-out), directly violating ADR 0009's locked opt-in default. Freshly-onboarded users scan everything.
2. **\*\*Medium\*\***: `/channels`` does not upsert candidates from `get_dialogs()` (ADR lines 48-54), so channels with no received traffic are undiscoverable/unwatchable.
3. **\*\*Low\*\***: `/watch <#name|id>` can't resolve by channel name/username, only numeric id (ADR line 64).

The primary finding is the ADR-contract violation on the live ingestion path affecting every user ? I'll rate it blocker since the entire selective-scanning feature is defeated for new channels and a fresh user gets the opposite of the ADR's guarantee.

**33. user (2026-07-06T09:13:45.137Z)**

Structured output provided successfully